

MCDI504

MACHINE LEARNING I

**AVANCE DEL PROYECTO — FASE 3:
DESARROLLO DEL PROYECTO**

Nombre integrante/s: Patricio Núñez, Jennifer Nilo

Curso: MCDI504 - Machine Learning I

Fecha: 20 agosto 2026

Grupo: 8

I. Índice

Contenido

I.	Índice.....	2
II.	Descripción del conjunto de datos y preparación para clasificación	3
2.1	Variables predictoras seleccionadas	3
2.2	Preparación analítica y partición de datos.....	4
III.	Análisis exploratorio inicial del conjunto de datos	6
3.1	Distribución de clases.....	6
3.2	Calidad de datos y valores faltantes.....	6
3.3	Patrones descriptivos relacionados con la clasificación.....	7
IV.	Notebook F3_Clasificacion.ipynb y documentación del proceso	9
4.1	Correspondencia con el repositorio	9
4.2	Evidencia resumida del flujo de código	9
V.	Modelo KNN	11
5.1	Configuración e implementación	11
5.2	Resultados e interpretación	12
VI.	Modelo de árboles de decisión	13
6.1	Configuración e implementación	13
6.2	Resultados e interpretación	14
VII.	Modelo SVM.....	15
7.1	Configuración e implementación	15
7.2	Resultados e interpretación	15
VIII.	Modelo Naive Bayes	16
8.1	Configuración e implementación	16
8.2	Resultados e interpretación	16
IX.	Modelo de redes neuronales	17
9.1	Configuración e implementación	17
9.2	Resultados e interpretación	17
X.	Evaluación preliminar de modelos de clasificación	19
10.1	Comparación analítica inicial.....	21
XI.	Conclusiones.....	23
XII.	Uso de fuentes, citación y referencias	24
XIII.	Anexos	25
	Anexo A. Evidencia resumida del código fuente	25
	Anexo B. Hiperparámetros y validación cruzada.....	25
	Anexo C. Matrices de confusión y resultados complementarios.....	26

II. Descripción del conjunto de datos y preparación para clasificación

El proyecto aborda un problema de clasificación supervisada binaria utilizando el conjunto de datos Titanic disponible en seaborn. La variable objetivo es `survived`, codificada como 0 para pasajeros que no sobrevivieron y 1 para pasajeros que sobrevivieron. El objetivo analítico consiste en estimar esa clase a partir de características demográficas, familiares y asociadas al viaje, manteniendo una separación clara entre información predictora e información que podría revelar directamente el resultado. Esta formulación es coherente con el enfoque de clasificación supervisada, donde el modelo aprende a partir de observaciones etiquetadas y posteriormente se evalúa sobre datos no utilizados durante el ajuste (James et al., 2021, Pedregosa et al., 2011).

La carga inicial contiene 891 observaciones y 15 columnas. Antes del modelamiento se revisan tipos de datos, ausencia de información y redundancia entre variables. Se excluyen `class`, `who`, `adult_male`, `embark_town` y `alone` por ser variables derivadas o redundantes, `alive` se descarta porque reproduce directamente la condición de supervivencia y generaría fuga de información, `deck` no se utiliza debido a su proporción elevada de valores faltantes. Como resultado, el modelamiento conserva siete variables predictoras originales las cuales son `pclass`, `sex`, `age`, `sibsp`, `parch`, `fare` y `embarked`.

Elemento	Descripción aplicada al caso
Dataset	Titanic, conjunto tabular con información de pasajeros.
Observaciones iniciales	891 registros.
Columnas iniciales	15 variables en la carga original.
Variable objetivo	<code>survived</code> : 0 = no sobrevivió, 1 = sobrevivió.
Predictores seleccionados	<code>pclass</code> , <code>sex</code> , <code>age</code> , <code>sibsp</code> , <code>parch</code> , <code>fare</code> y <code>embarked</code> .
Propósito analítico	Comparar modelos supervisados para clasificar supervivencia.
Tipo de aprendizaje	Clasificación supervisada binaria.



Figura 1. Flujo analítico aplicado en la Fase 3.

2.1 Variables predictoras seleccionadas

La selección de variables busca conservar atributos disponibles antes del resultado de supervivencia y evitar columnas que dupliquen información. `pclass` y `fare` aportan información asociada a la clase de viaje y al valor del pasaje, `age` y `sex` representan características demográficas, `sibsp` y `parch` describen la composición familiar a bordo, `embarked` identifica el puerto de embarque. Las variables categóricas se transforman mediante one-hot encoding dentro del pipeline, por lo que el espacio final utilizado por los modelos contiene cinco variables numéricas y cinco indicadores categóricos, para un total de diez columnas después de la transformación.

Variable	Rol	Tratamiento en el pipeline	Descripción
pclass	Predictora numérica	Imputación si corresponde, escalamiento en KNN, SVM y MLP.	Clase del pasaje.
age	Predictora numérica	Imputación con mediana, escalamiento en KNN, SVM y MLP.	Edad del pasajero.
sibsp	Predictora numérica	Escalamiento en KNN, SVM y MLP.	Hermanos o cónyuges a bordo.
parch	Predictora numérica	Escalamiento en KNN, SVM y MLP.	Padres o hijos a bordo.
fare	Predictora numérica	Escalamiento en KNN, SVM y MLP.	Tarifa pagada.
sex	Predictora categórica	Imputación por moda y one-hot encoding.	Sexo registrado.
embarked	Predictora categórica	Imputación por moda y one-hot encoding.	Puerto de embarque.
survived	Objetivo	No se transforma como predictor.	Clase de supervivencia.

2.2 Preparación analítica y partición de datos

El preprocesamiento se implementa con Pipeline y ColumnTransformer para que la imputación, la codificación y el escalamiento se ajusten únicamente con los datos de entrenamiento. Este diseño reduce el riesgo de fuga de información, ya que las estadísticas utilizadas para completar o escalar variables no se calculan a partir del conjunto de prueba. Las variables numéricas se imputan con la mediana, sex y embarked se imputan con la categoría más frecuente y posteriormente se codifican con OneHotEncoder. KNN, SVM y las redes neuronales incorporan StandardScaler en las variables numéricas, mientras que el árbol de decisión y Gaussian Naive Bayes utilizan el preprocesamiento sin escalamiento.

Acción	Implementación	Justificación
Selección de variables	7 predictores originales	Reduce redundancia y evita fuga de información.
Imputación numérica	SimpleImputer(strategy="median")	Conserva registros con age faltante usando una medida resistente a valores extremos.
Imputación categórica	SimpleImputer(strategy="most_frequent")	Completa categorías faltantes de embarked.
Codificación	OneHotEncoder(handle_unknown="ignore")	Convierte categorías a indicadores binarios y permite manejar categorías no vistas.
Partición	test_size=0.20, random_state=42, stratify=y	Reserva 20% para prueba y mantiene la proporción de clases.
Validación	StratifiedKFold(n_splits=5, shuffle=True, random_state=42)	Evalúa configuraciones en cinco particiones preservando la distribución de clases.
Criterio de búsqueda	GridSearchCV(scoring="f1")	Selecciona hiperparámetros priorizando el equilibrio entre precision y recall.

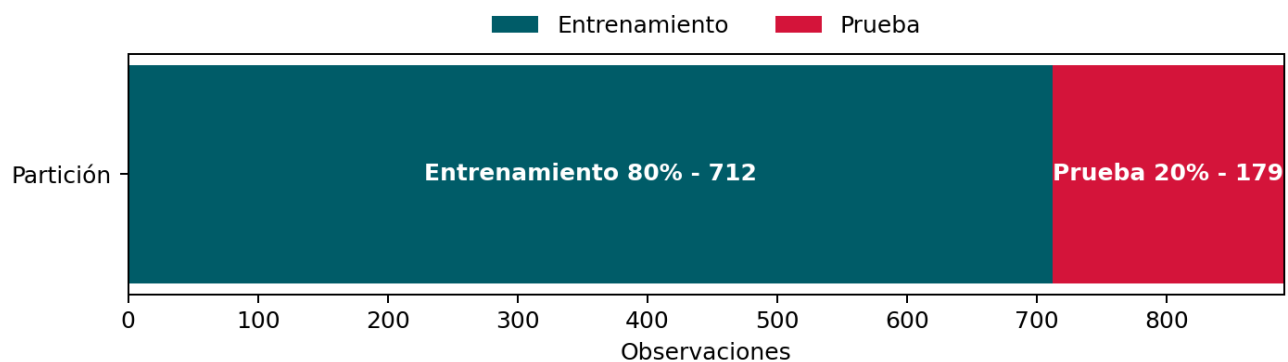


Figura 2. Partición estratificada de entrenamiento y prueba.

La partición genera 712 observaciones de entrenamiento y 179 de prueba. La clase positiva representa 38.34% del entrenamiento y 38.55% de la prueba, por lo que ambos conjuntos conservan proporciones similares de la variable objetivo.

III. Análisis exploratorio inicial del conjunto de datos

El análisis exploratorio se utiliza para caracterizar la variable objetivo, revisar la calidad de los datos e identificar patrones que orienten el modelamiento. Esta etapa no reemplaza la evaluación predictiva, pero permite anticipar aspectos relevantes como desbalance de clases, ausencia de valores y diferencias descriptivas entre grupos. En problemas de clasificación, estas observaciones ayudan a justificar la preparación de los datos y la selección de métricas más allá del accuracy (James et al., 2021).

3.1 Distribución de clases

La variable *survived* presenta una clase mayoritaria de no sobrevivientes, con 549 registros, y una clase positiva de 342 sobrevivientes. La proporción de la clase positiva corresponde a 38,38%, por lo que existe una diferencia entre clases, aunque ambas mantienen una representación suficiente para el análisis. Debido a esta distribución, el informe considera *precision*, *recall*, *F1-score* y *ROC-AUC* junto con *accuracy*, evitando interpretar el desempeño únicamente a partir del porcentaje total de aciertos.

Clase	Codificación	Frecuencia	Porcentaje
No sobrevivió	0	549	61,62%
Sobrevivió	1	342	38,38%
Total	-	891	100,00%

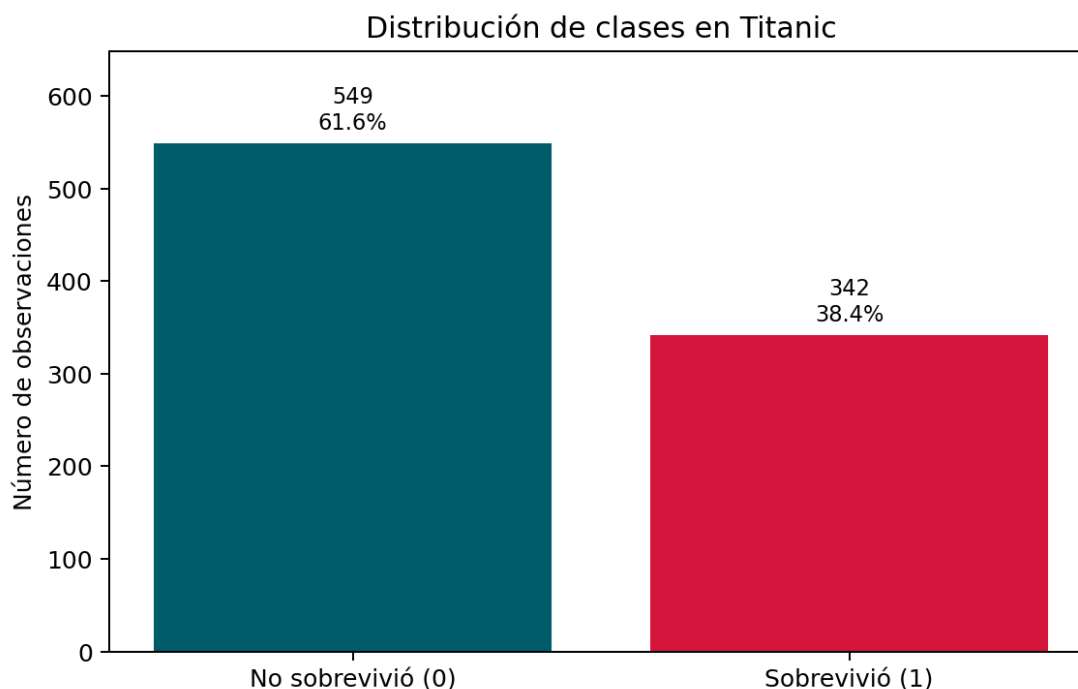


Figura 3. Distribución general de clases en el dataset Titanic.

3.2 Calidad de datos y valores faltantes

La revisión de calidad identifica ausencia de información principalmente en *age*, *embarked* y *deck*. La variable *age* contiene 177 valores faltantes y *embarked* presenta 2 registros sin categoría, por lo que ambas variables se mantienen y se imputan dentro del pipeline. La variable *deck* registra 688 valores faltantes, equivalentes a aproximadamente tres cuartas partes del conjunto de datos y además no resulta indispensable para el objetivo definido, por esta razón se excluye del conjunto de predictores. Esta decisión

permite conservar el mayor número de observaciones sin introducir una variable con información ampliamente incompleta.

Variable	Valores faltantes	Porcentaje	Tratamiento
age	177	19,87%	Imputación con mediana dentro del pipeline.
embarked	2	0,22%	Imputación con categoría más frecuente.
deck	688	77,22%	Exclusión por alta ausencia y baja prioridad analítica.

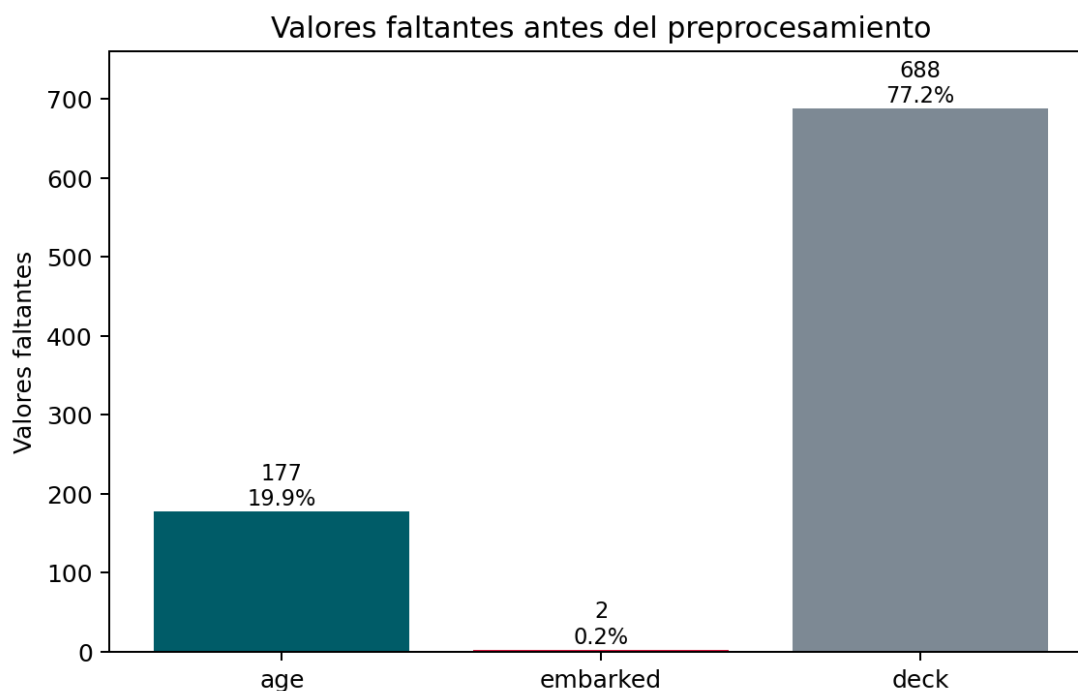


Figura 4. Valores faltantes relevantes antes del preprocesamiento.

3.3 Patrones descriptivos relacionados con la clasificación

La exploración muestra diferencias descriptivas relevantes en sex y pclass. La tasa observada de supervivencia es 74.2% para mujeres y 18.9% para hombres. Por clase del pasaje, la supervivencia alcanza 63.0% en primera clase, 47.3% en segunda y 24.2% en tercera. Estos patrones justifican la inclusión de ambas variables, pero se interpretan como asociaciones descriptivas dentro del dataset y no como relaciones causales.

Patrones descriptivos de supervivencia

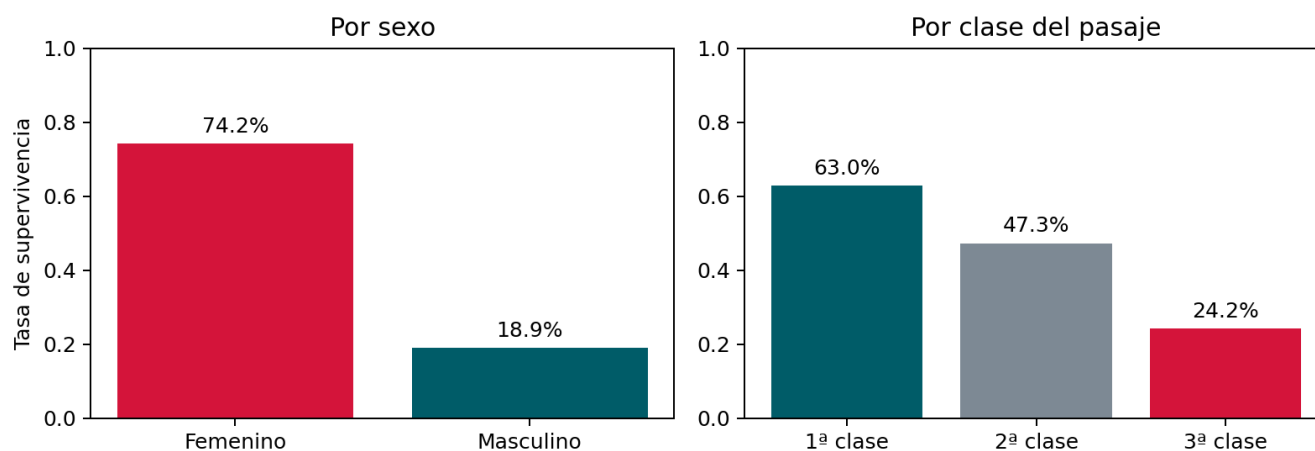


Figura 5. Tasas descriptivas de supervivencia por sexo y clase del pasaje.

En conjunto, el EDA respalda la formulación del problema de clasificación y la estrategia de preparación utilizada. La distribución de la variable objetivo justifica una partición estratificada, la ausencia de valores fundamenta el uso de imputadores dentro del pipeline, y las diferencias observadas en variables como sex y pclass sugieren que existe señal predictiva aprovechable por modelos con mecanismos distintos. La validez de esa señal se contrasta posteriormente mediante validación cruzada y evaluación sobre el conjunto de prueba.

IV. Notebook F3_Clasificacion.ipynb y documentación del proceso

El notebook constituye la evidencia técnica principal de la Fase 3 y organiza el trabajo en una secuencia reproducible, carga y descripción de datos, EDA, selección de variables, partición entrenamiento/prueba, definición de preprocesadores, búsqueda de hiperparámetros, evaluación individual y comparación final. La implementación utiliza scikit-learn y concentra el preprocesamiento junto al estimador mediante pipelines, lo que permite que cada modelo reciba las transformaciones definidas sin aplicar información del conjunto de prueba durante el ajuste (Pedregosa et al., 2011).

Cada modelo se selecciona con GridSearchCV utilizando F1 como criterio, validación cruzada estratificada de cinco particiones y posterior refit sobre el conjunto completo de entrenamiento. Una vez seleccionado el estimador, se calculan métricas en prueba, matriz de confusión y puntajes continuos para ROC-AUC.

Etapa	Código o acción principal	Evidencia	Sección del informe
Carga	<code>sns.load_dataset("titanic")</code>	Dimensiones, primeras filas y tipos.	II-III
Selección	<code>selected_features</code>	Siete predictores sin variables de fuga.	II
Partición	<code>train_test_split(..., stratify=y)</code>	712 filas de entrenamiento y 179 de prueba.	II
Preprocesamiento	Pipeline + ColumnTransformer	Imputación, one-hot encoding y escalamiento.	II-III
Validación	GridSearchCV + StratifiedKFold	Mejor configuración por F1 en 5 folds.	V-IX
Evaluación	accuracy, precision, recall, F1, ROC-AUC, matriz	Resultados comparables en prueba.	V-X
Exportación	<code>resultados_fase3/</code>	Tablas y figuras para anexos.	XIII

4.1 Correspondencia con el repositorio

Para conservar la trazabilidad del proyecto, el repositorio contiene el notebook ejecutable, la descripción del conjunto de datos y el informe final. El README indica dependencias, instrucciones de ejecución y la ubicación de los resultados exportados.

La URL del repositorio es: “<https://github.com/pnunezcarreno/machinelearning1>”.

Elemento sugerido	Contenido
<code>/notebook/F3_Clasificacion.ipynb</code>	Notebook ejecutable con narrativa, código y resultados.
<code>/docs/</code>	Informe final PDF.
README.md	Dependencias, ejecución y descripción del flujo.

4.2 Evidencia resumida del flujo de código

La partición estratificada, el preprocesamiento dentro de Pipeline y la validación cruzada con StratifiedKFold se definen de forma explícita en el notebook. Los siete modelos utilizan la misma separación de datos y la

misma función de evaluación, por lo que las diferencias reportadas se relacionan con la configuración seleccionada y el comportamiento de cada clasificador, no con cambios en el conjunto de prueba.

V. Modelo KNN

k-Nearest Neighbors clasifica una observación a partir de la clase predominante entre sus vecinos más próximos. El parámetro k controla el tamaño del vecindario y la métrica de distancia determina cómo se mide la proximidad, además, el escalamiento resulta relevante porque variables con magnitudes mayores pueden dominar la distancia si no se normalizan (Ruete, 2026a, Scikit-learn developers, n.d.-a). En este proyecto, el modelo se integra con StandardScaler para las variables numéricas y la selección de hiperparámetros se realiza mediante validación cruzada.

5.1 Configuración e implementación

```
knn_pipeline = Pipeline([
    ("prep", preprocessor_scaled),
    ("model", KNeighborsClassifier())
])

knn_grid = {
    "model__n_neighbors": [3, 5, 7, 9, 11, 15],
    "model__weights": ["uniform", "distance"],
    "model__p": [1, 2]
}
```

Parámetro	Valor seleccionado	Interpretación
n_neighbors	3	Número de vecinos considerados en la clasificación.
weights	uniform	Todos los vecinos seleccionados aportan el mismo peso.
p	1	$p=1$ corresponde a distancia Manhattan.
Escalamiento	StandardScaler en variables numéricas	Evita dominancia por diferencias de escala.
Criterio de selección	F1 en 5-fold CV	Prioriza equilibrio entre precision y recall.

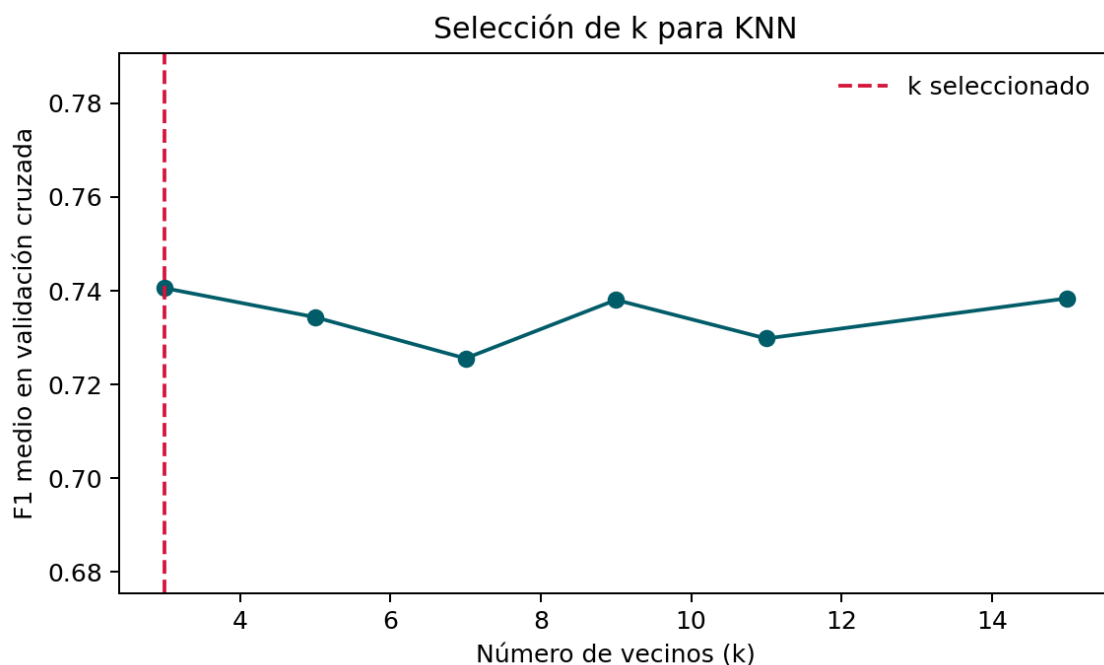


Figura 6. F1 medio de validación cruzada según el número de vecinos.

5.2 Resultados e interpretación

Métrica / resultado	Valor
F1 validación cruzada	0,7406
Accuracy prueba	0,8156
Precision clase positiva	0,7727
Recall clase positiva	0,7391
F1-score clase positiva	0,7556
ROC-AUC	0,8531
Matriz de confusión	[[95, 15], [18, 51]]

KNN alcanza un accuracy de 0,8156 y un F1 de 0,7556 en el conjunto de prueba. La matriz registra 51 verdaderos positivos y 18 falsos negativos, con un recall de 0,7391 para la clase sobrevivió. Su ROC-AUC de 0,8531 es uno de los valores más altos del conjunto evaluado, lo que indica una capacidad útil para ordenar observaciones según probabilidad relativa de pertenecer a la clase positiva.

La comparación entre F1 de entrenamiento (0,8291) y F1 medio de validación (0,7406) muestra una brecha de 0,0886. El resultado sugiere que el modelo captura patrones del entrenamiento con mayor facilidad que de las particiones de validación, por lo que su configuración debe interpretarse junto con el desempeño de prueba. Aun así, la incorporación de escalamiento y selección de k mediante validación cruzada evita depender de una configuración arbitraria.

VI. Modelo de árboles de decisión

El árbol de decisión construye reglas jerárquicas que dividen el espacio de predictores en regiones cada vez más homogéneas respecto de la clase objetivo. Su principal ventaja analítica es que las decisiones pueden examinarse como una secuencia de umbrales, aunque una profundidad alta puede aumentar el ajuste al ruido del entrenamiento. Por este motivo, la profundidad y el tamaño mínimo de las hojas se consideran hiperparámetros relevantes para controlar la complejidad (Ruete, 2026b, Scikit-learn developers, n.d.-b).

6.1 Configuración e implementación

```
tree_pipeline = Pipeline([
    ("prep", preprocessor_unscaled),
    ("model", DecisionTreeClassifier(random_state=42))
])

tree_grid = {
    "model__criterion": ["gini", "entropy"],
    "model__max_depth": [3, 4, 5, 6, 8],
    "model__min_samples_leaf": [1, 3, 5, 10]
}
```

Parámetro	Valor seleccionado	Interpretación
criterion	entropy	Criterio utilizado para evaluar la calidad de las divisiones.
max_depth	8	Profundidad máxima seleccionada por validación.
min_samples_leaf	1	Mínimo de observaciones permitido en una hoja.
Escalamiento	No aplicado	Los árboles no dependen de distancias entre variables.

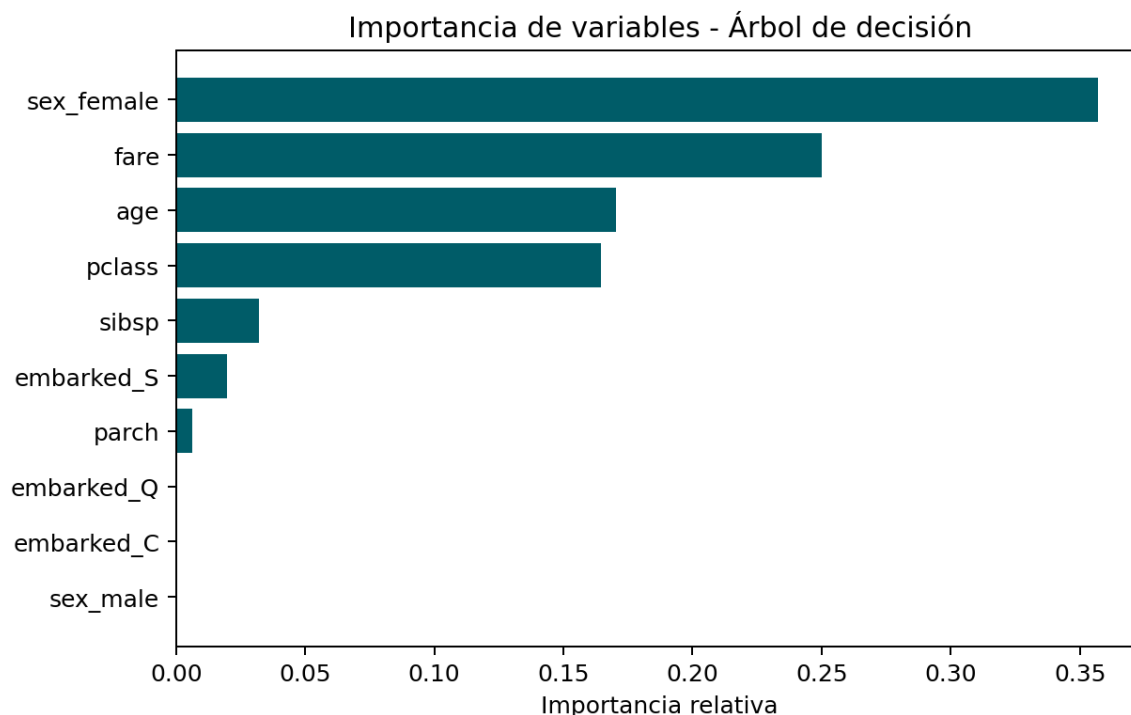


Figura 7. Importancia relativa de variables en el árbol seleccionado.

6.2 Resultados e interpretación

Métrica / resultado	Valor
F1 validación cruzada	0,744
Accuracy prueba	0,7933
Precision clase positiva	0,7963
Recall clase positiva	0,6232
F1-score clase positiva	0,6992
ROC-AUC	0,8073
Matriz de confusión	[[99, 11], [26, 43]]

El árbol obtiene $\text{accuracy}=0,7933$ y $F1=0,6992$. Su precision de 0,7963 es alta en relación con su recall de 0,6232, esto se observa en 26 falsos negativos frente a 11 falsos positivos. La importancia de variables concentra una proporción relevante en atributos como sex, pclass y fare, lo que coincide con los patrones descriptivos observados en el EDA, aunque la importancia del árbol representa uso predictivo dentro del modelo y no una interpretación causal.

La brecha entre F1 de entrenamiento (0,8600) y validación (0,7440) es 0,1160 y es mayor que la observada en SVM. Este patrón es consistente con la capacidad de los árboles para ajustarse con detalle a particiones específicas cuando la profundidad aumenta. El control de `max_depth` y `min_samples_leaf` limita parte de esa complejidad.

VII. Modelo SVM

Las máquinas de vectores de soporte buscan una frontera de decisión que separe las clases maximizando el margen entre observaciones críticas. El uso de un kernel RBF permite representar fronteras no lineales mediante una transformación implícita del espacio de características, mientras que C controla el equilibrio entre margen y penalización por errores. Debido a que la geometría del espacio es central en este método, el escalamiento de variables numéricas forma parte del pipeline de preparación (Ruete, 2026c, Scikit-learn developers, n.d.-c).

7.1 Configuración e implementación

```
svm_pipeline = Pipeline([
    ("prep", preprocessor_scaled),
    ("model", SVC())
])

svm_grid = [
    {"model__kernel": ["linear"], "model__C": [0.1, 1, 10]},
    {"model__kernel": ["rbf"], "model__C": [0.1, 1, 10],
     "model__gamma": ["scale", "auto"]}
]
```

Parámetro	Valor seleccionado	Interpretación
kernel	rbf	Kernel RBF para separación no lineal.
C	1	Controla la penalización asociada a errores de clasificación.
gamma	auto	Controla el alcance de influencia de las observaciones en RBF.
Escalamiento	StandardScaler en variables numéricas	Homogeneiza la escala usada para construir la frontera.

7.2 Resultados e interpretación

Métrica / resultado	Valor
F1 validación cruzada	0,7558
Accuracy prueba	0,8268
Precision clase positiva	0,8167
Recall clase positiva	0,7101
F1-score clase positiva	0,7597
ROC-AUC	0,8411
Matriz de confusión	[[99, 11], [20, 49]]

SVM obtiene el mayor accuracy del conjunto evaluado, con 0,8268, y también el mayor F1 en prueba, con 0,7597. La matriz de confusión contiene 49 verdaderos positivos, 99 verdaderos negativos, 11 falsos positivos y 20 falsos negativos. El recall de 0,7101 es inferior al de KNN y MLP de dos capas, pero la precision de 0,8167 compensa parte de esa diferencia y produce el mejor equilibrio global medido por F1.

La diferencia entre F1 de entrenamiento (0,7692) y validación (0,7558) es de solo 0,0134. Esta brecha es la menor entre los modelos con diferencias positivas relevantes, lo que sugiere un comportamiento más estable entre particiones de validación. Su limitación principal en este contexto no está en el desempeño, sino en la menor facilidad para explicar decisiones individuales en comparación con un árbol de decisión.

VIII. Modelo Naive Bayes

Gaussian Naive Bayes es un clasificador probabilístico que aplica el teorema de Bayes y asume independencia condicional entre predictores una vez conocida la clase. Para variables continuas, la variante gaussiana modela la distribución de cada predictor por clase mediante una distribución normal. El supuesto simplifica el cálculo y permite un entrenamiento eficiente, aunque puede ser restrictivo cuando existen correlaciones entre variables del conjunto de datos (Ruete, 2026d, Scikit-learn developers, n.d.-d).

8.1 Configuración e implementación

```
nb_pipeline = Pipeline([
    ("prep", preprocessor_unscaled),
    ("model", GaussianNB())
])

nb_grid = {
    "model__var_smoothing": np.logspace(-11, -7, 5)
}
```

Parámetro / supuesto	Valor seleccionado	Interpretación
Variante	GaussianNB	Adecuada para predictores continuos modelados por distribución normal por clase.
var_smoothing	1e-11	Pequeño término de estabilización aplicado a las varianzas.
priors	No especificado	Las probabilidades previas se estiman desde los datos.
Escalamiento	No aplicado	No es requisito del modelo, se mantienen valores imputados y variables categóricas codificadas.

8.2 Resultados e interpretación

Métrica / resultado	Valor
F1 validación cruzada	0,7355
Accuracy prueba	0,7877
Precision clase positiva	0,7385
Recall clase positiva	0,6957
F1-score clase positiva	0,7164
ROC-AUC	0,8191
Matriz de confusión	[[93, 17], [21, 48]]

Naive Bayes alcanza accuracy=0,7877 y F1=0,7164. El modelo identifica 48 verdaderos positivos y registra 21 falsos negativos, con recall=0,6957. Su ROC-AUC de 0,8191 queda por debajo de KNN, SVM y las redes neuronales, pero mantiene un rendimiento que permite utilizarlo como referencia probabilística dentro de la comparación.

El F1 medio de validación (0,7355) es levemente superior al F1 medio de entrenamiento (0,7304) en las particiones evaluadas. La diferencia es pequeña y no constituye evidencia de una ventaja estructural, pero sí muestra que el modelo no presenta una brecha de entrenamiento positiva como la observada en el árbol o en las redes más profundas. Su principal limitación sigue siendo el supuesto de independencia condicional, que no necesariamente refleja las relaciones entre variables como pclass, fare y otras características del viaje.

IX. Modelo de redes neuronales

El perceptrón multicapa (MLP) organiza la información en una capa de entrada, una o más capas ocultas y una capa de salida. Cada capa combina entradas mediante pesos y sesgos y aplica una función de activación no lineal, lo que permite representar relaciones que no se describen adecuadamente mediante una frontera lineal simple. La profundidad aumenta la capacidad representacional, pero también puede incrementar el costo de ajuste y la diferencia entre desempeño de entrenamiento y validación si la arquitectura supera lo necesario para los datos disponibles (Ruete, 2026e, Géron, 2019, Scikit-learn developers, n.d.-e).

Se evaluaron tres arquitecturas con una, dos y tres capas ocultas para analizar el efecto de la profundidad manteniendo constantes los principales parámetros de entrenamiento. Todas emplean `activation="relu"`, `solver="adam"`, `max_iter=500` y `random_state=42`, y reciben el mismo preprocesamiento escalado utilizado por KNN y SVM. La comparación no busca asumir que una red más profunda sea mejor, sino observar si la complejidad adicional se traduce en mejoras consistentes en validación y prueba.

9.1 Configuración e implementación

```
MLPClassifier(hidden_layer_sizes=(100,), activation='relu', solver='adam', max_iter=500, random_state=42)
MLPClassifier(hidden_layer_sizes=(100, 50), activation='relu', solver='adam', max_iter=500, random_state=42)
MLPClassifier(hidden_layer_sizes=(100, 50, 25), activation='relu', solver='adam', max_iter=500, random_state=42)
```

Arquitectura	hidden_layer_sizes	activation	solver	max_iter	n_iter_
MLP 1 capa oculta	-100	relu	adam	500	500
MLP 2 capas ocultas	(100, 50)	relu	adam	500	485
MLP 3 capas ocultas	(100, 50, 25)	relu	adam	500	296

9.2 Resultados e interpretación

Modelo	Accuracy	Precision	Recall	F1	ROC-AUC	Matriz de confusión
MLP 1 capa oculta	0,7877	0,7818	0,6232	0,6935	0,8539	[[98, 12], [26, 43]]
MLP 2 capas ocultas	0,7933	0,7353	0,7246	0,7299	0,853	[[92, 18], [19, 50]]
MLP 3 capas ocultas	0,7989	0,7619	0,6957	0,7273	0,858	[[95, 15], [21, 48]]

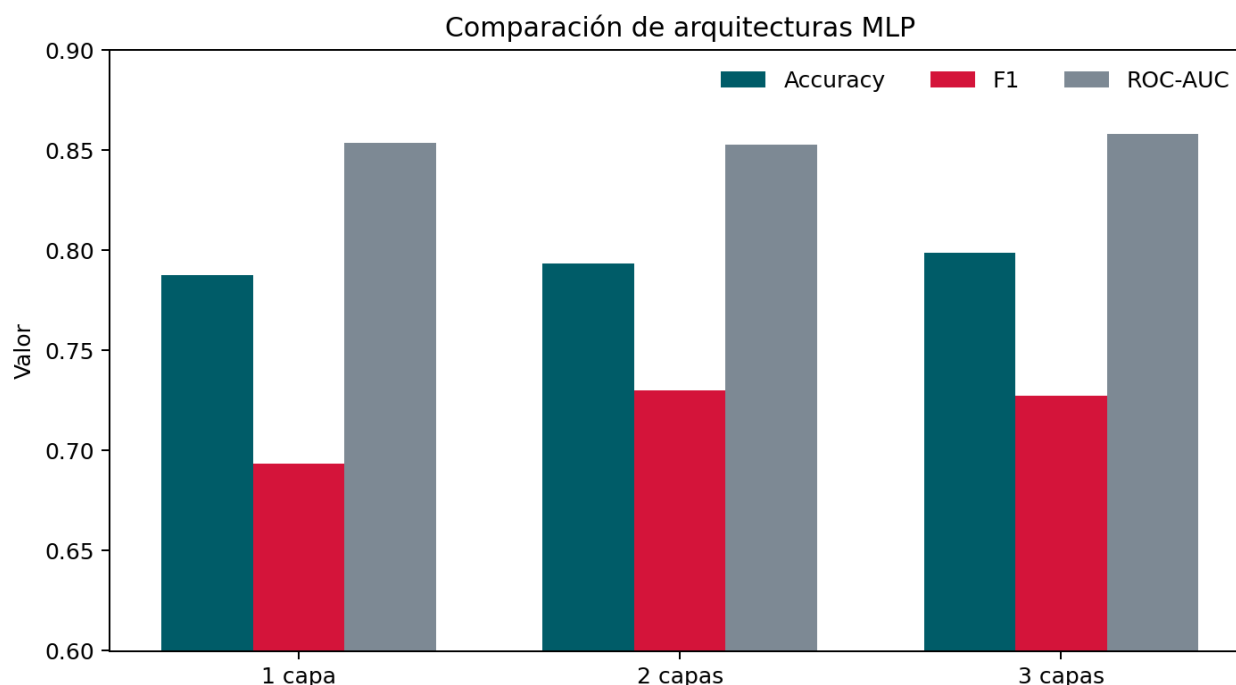


Figura 8. Comparación de métricas entre las tres arquitecturas MLP.

La red con una capa obtiene $\text{accuracy}=0,7877$ y $\text{F1}=0,6935$, además, alcanza las 500 iteraciones configuradas, por lo que su convergencia debe revisarse antes de ampliar el análisis. La red con dos capas mejora el F1 a 0,7299 y alcanza el mayor recall entre las tres redes, con 0,7246. La red con tres capas logra $\text{accuracy}=0,7989$ y $\text{F1}=0,7273$, mientras que su ROC-AUC de 0,8580 es el mayor entre los siete modelos evaluados.

La profundidad adicional no produce una mejora uniforme. La brecha F1 entrenamiento-validación aumenta desde 0,0660 en una capa a 0,1371 en dos y 0,1589 en tres capas. Este patrón indica que las redes más profundas ajustan mejor el entrenamiento, pero esa ganancia no se conserva en validación con la misma magnitud. Para este conjunto tabular y tamaño de muestra, la comparación favorece una lectura centrada en generalización y no únicamente en capacidad del modelo.

X. Evaluación preliminar de modelos de clasificación

La evaluación final utiliza el mismo conjunto de prueba para los siete modelos y combina métricas que describen aspectos distintos del error. Accuracy resume la proporción total de aciertos, precision indica qué proporción de las predicciones positivas corresponde realmente a sobrevivientes, recall mide qué proporción de sobrevivientes fue identificada, F1 combina precision y recall y ROC-AUC evalúa la capacidad de discriminación a partir de un puntaje continuo. Considerar varias métricas permite evitar conclusiones basadas únicamente en la clase mayoritaria o en un único tipo de error (James et al., 2021).

Modelo	Accuracy	Precision	Recall	F1	ROC-AUC	VN/FP/FN/VP
SVM	0,8268	0,8167	0,7101	0,7597	0,8411	99/11/20/49
KNN	0,8156	0,7727	0,7391	0,7556	0,8531	95/15/18/51
MLP 2 capas ocultas	0,7933	0,7353	0,7246	0,7299	0,853	92/18/19/50
MLP 3 capas ocultas	0,7989	0,7619	0,6957	0,7273	0,858	95/15/21/48
Naive Bayes	0,7877	0,7385	0,6957	0,7164	0,8191	93/17/21/48
Árbol de decisión	0,7933	0,7963	0,6232	0,6992	0,8073	99/11/26/43
MLP 1 capa oculta	0,7877	0,7818	0,6232	0,6935	0,8539	98/12/26/43

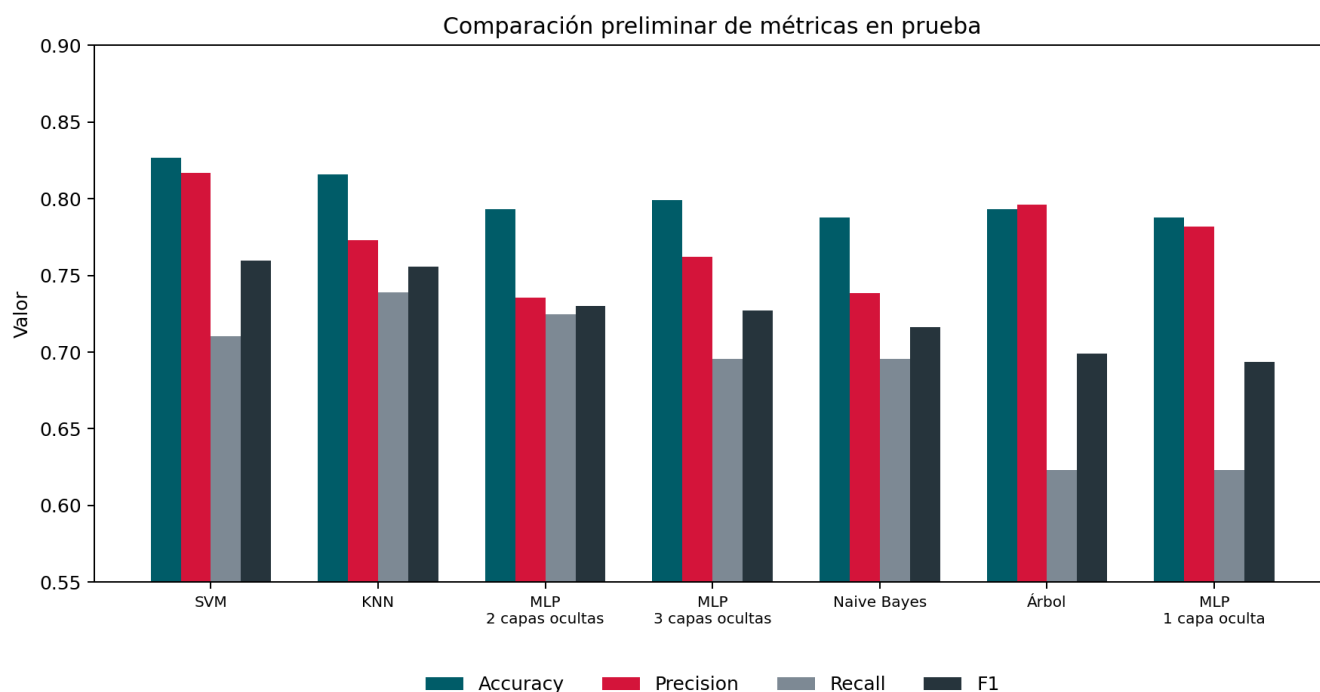


Figura 9. Comparación de accuracy, precision, recall y F1 en el conjunto de prueba.

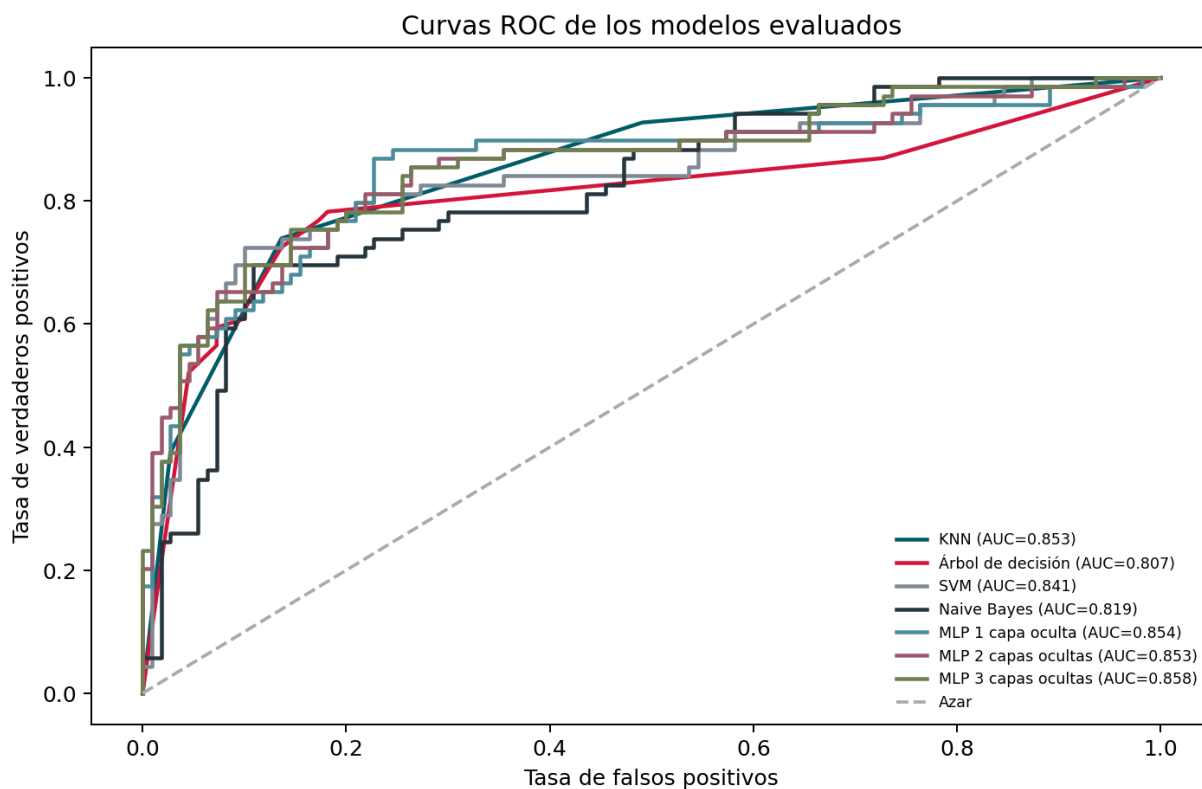


Figura 10. Curvas ROC de los siete modelos evaluados.

Matrices de confusión - conjunto de prueba

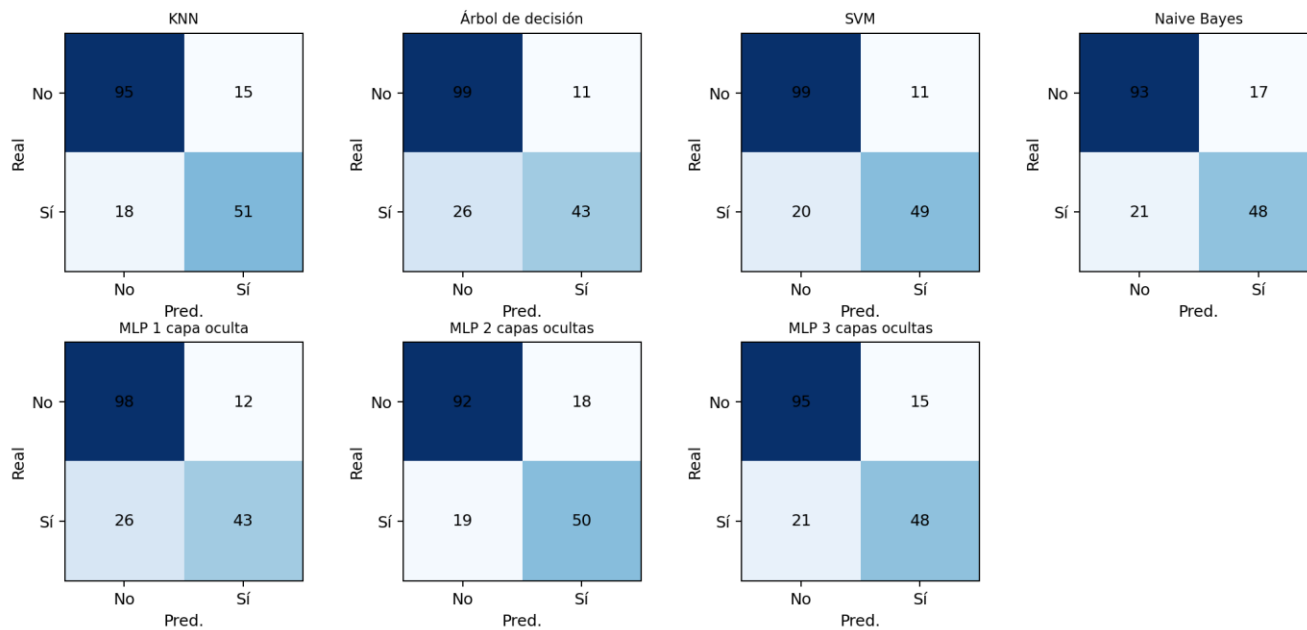


Figura 11. Matrices de confusión de los siete modelos en el conjunto de prueba.

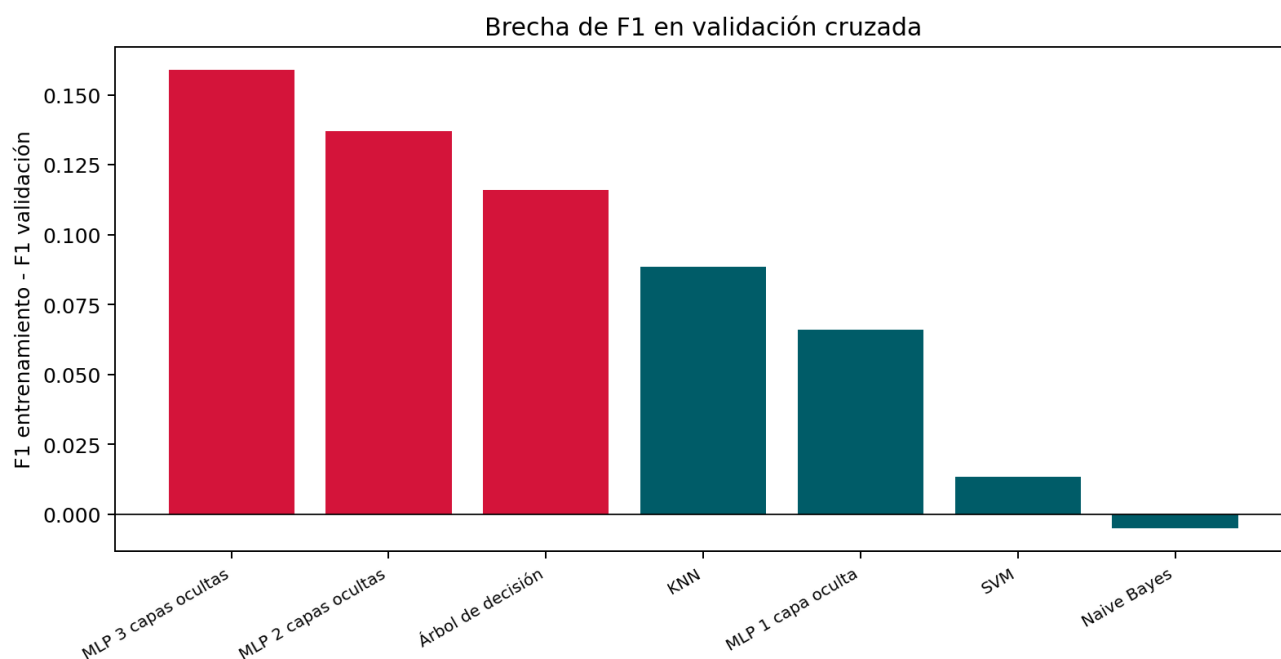


Figura 12. Brecha entre F1 de entrenamiento y validación cruzada.

10.1 Comparación analítica inicial

Modelo	Fortaleza observada	Limitación observada	Lectura del resultado
KNN	Buen ROC-AUC y recall competitivo, método basado en proximidad.	Brecha visible entre entrenamiento y validación, sensible a elección de k y escala.	Segundo F1 más alto y buen desempeño general.
Árbol de decisión	Interpretabilidad y lectura de importancia de variables.	Mayor brecha de generalización que SVM, recall más bajo del grupo.	Útil cuando la explicación mediante reglas es prioritaria.
SVM	Mayor accuracy y F1, brecha CV pequeña.	Menor interpretabilidad directa y dependencia de escalamiento/hiperparámetros.	Mejor equilibrio global en esta ejecución.
Naïve Bayes	Modelo probabilístico eficiente y con brecha CV mínima.	Supuesto de independencia condicional restrictivo.	Referencia probabilística útil, sin liderar métricas de prueba.
MLP 1 capa	Arquitectura simple y ROC-AUC alto.	Alcanza max_iter=500 y obtiene el F1 más bajo de las redes.	Requiere revisar convergencia y regularización.
MLP 2 capas	Mayor recall y F1 entre las redes.	Brecha de entrenamiento-validación superior a una capa.	Mejor equilibrio dentro de las arquitecturas MLP.
MLP 3 capas	Mayor ROC-AUC de todos los modelos.	Mayor brecha CV del conjunto evaluado.	Buena discriminación, pero sin superar a SVM en accuracy o F1.

La comparación muestra que SVM lidera tanto accuracy (0,8268) como F1 (0,7597), mientras que MLP con tres capas obtiene el ROC-AUC más alto (0,8580). El mejor recall corresponde a KNN, con 0,7391. Por tanto, no todas las métricas identifican al mismo modelo y la selección depende del objetivo, si se busca equilibrio entre precision y recall en la clase positiva, SVM presenta la mejor combinación, si se prioriza discriminación global de puntajes, MLP de tres capas obtiene la mayor AUC.

La validación cruzada agrega una segunda lectura. SVM mantiene una diferencia reducida entre entrenamiento y validación, mientras que el árbol y las redes de dos y tres capas presentan brechas mayores. Esta evidencia sugiere que el incremento de complejidad no se traduce automáticamente en una mejor capacidad de generalización. En consecuencia, el resultado preliminar favorece SVM como candidato principal para la continuidad del proyecto, manteniendo KNN y MLP como alternativas de comparación y el árbol como referencia interpretable.

XI. Conclusiones

La Fase 3 permitió implementar y documentar un flujo completo de clasificación supervisada sobre el dataset Titanic. El trabajo integra revisión de calidad, selección de siete variables predictoras, imputación y codificación dentro de pipelines, partición estratificada entrenamiento/prueba, validación cruzada y evaluación de siete modelos. La estructura utilizada mantiene el conjunto de prueba fuera del ajuste de hiperparámetros y permite relacionar las decisiones técnicas con evidencia cuantitativa obtenida por el código.

Entre los modelos tradicionales, SVM obtiene el mejor desempeño global con $\text{accuracy}=0,8268$ y $F1=0,7597$. KNN presenta un resultado cercano en F1 (0,7556) y el ROC-AUC más alto entre los modelos tradicionales (0,8531). El árbol de decisión aporta una ventaja de interpretabilidad, aunque su recall y su brecha entre entrenamiento y validación son menos favorables. Naive Bayes entrega un punto de comparación probabilístico con un desempeño intermedio y sin una brecha positiva relevante entre entrenamiento y validación.

En las redes neuronales, la arquitectura de dos capas obtiene el mayor F1 (0,7299) y recall (0,7246) dentro del grupo, mientras que la red de tres capas alcanza el mayor ROC-AUC general (0,8580). Sin embargo, las brechas de validación aumentan con la profundidad, por lo que no se observa evidencia suficiente para afirmar que una arquitectura más profunda sea una mejor elección para este problema. La red de una capa, además, alcanza el límite de 500 iteraciones configurado.

Como conclusión, SVM se mantiene como el candidato principal por combinar el mayor accuracy, el mayor F1 y una brecha reducida entre entrenamiento y validación.

Criterio de cierre	Resultado de la fase
Modelo con mayor accuracy	SVM (0,8268).
Modelo con mayor F1	SVM (0,7597).
Modelo con mayor ROC-AUC	MLP 3 capas (0,8580).
Modelo con mayor recall	KNN (0,7391).
Principal evidencia de generalización	SVM presenta una brecha F1 train-validación de 0,0134.

XII. Uso de fuentes, citación y referencias

Las decisiones conceptuales y metodológicas del informe se sustentan en bibliografía académica, documentación técnica oficial y recursos de la asignatura.

- Alpaydin, E. (2010). *Introduction to machine learning*. MIT Press.
- Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.)*. O'Reilly Media.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R (2nd ed.)*. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research*, 12, 2825-2830.
- Ruete, D. (2026a). *k-Nearest Neighbors (k-NN)* [Apunte]. Universidad Andrés Bello.
- Ruete, D. (2026b). *Árboles de decisión* [Apunte]. Universidad Andrés Bello.
- Ruete, D. (2026c). *Máquinas de Vectores de Soporte (SVM)* [Apunte]. Universidad Andrés Bello.
- Ruete, D. (2026d). *Naive Bayes* [Apunte]. Universidad Andrés Bello.
- Ruete, D. (2026e). *Redes Neuronales* [Apunte]. Universidad Andrés Bello.
- Scikit-learn developers. (n.d.-a). *KNeighborsClassifier*. *Scikit-learn*. Retrieved August 20, 2026, from <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>
- Scikit-learn developers. (n.d.-b). *DecisionTreeClassifier*. *Scikit-learn*. Retrieved August 20, 2026, from <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- Scikit-learn developers. (n.d.-c). *SVC*. *Scikit-learn*. Retrieved August 20, 2026, from <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- Scikit-learn developers. (n.d.-d). *GaussianNB*. *Scikit-learn*. Retrieved August 20, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html
- Scikit-learn developers. (n.d.-e). *MLPClassifier*. *Scikit-learn*. Retrieved August 20, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
- Waskom, M. L. (2021). *seaborn: Statistical data visualization*. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>

XIII. Anexos

Anexo A. Evidencia resumida del código fuente

Los siguientes fragmentos sintetizan las secciones que producen las evidencias principales del informe. Los fragmentos se presentan únicamente como apoyo de trazabilidad y no reemplazan el archivo ejecutable.

```
selected_features = ["pclass", "sex", "age", "sibsp", "parch", "fare", "embarked"]
X = titanic_df[selected_features].copy()
y = titanic_df["survived"].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

numeric_scaled = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])
```

Estos bloques evidencian que la selección de variables, la partición estratificada y las transformaciones se definen antes del ajuste de los clasificadores. La estructura del pipeline permite aplicar las mismas reglas de preparación en entrenamiento, validación y prueba, evitando transformaciones manuales separadas que podrían generar inconsistencias entre modelos.

Anexo B. Hiperparámetros y validación cruzada

Modelo	Mejores hiperparámetros	F1 train CV	F1 validación CV	Brecha
KNN	n_neighbors=3, p=1, weights=uniform	0,8291	0,7406	0,0886
Árbol de decisión	criterion=entropy, max_depth=8, min_samples_leaf=1	0,86	0,744	0,116
SVM	C=1, gamma=auto, kernel=rbf	0,7692	0,7558	0,0134
Naive Bayes	var_smoothing=1e-11	0,7304	0,7355	-0,0051
MLP 1 capa oculta	activation=relu, hidden_layer_sizes=(100,), max_iter=500, solver=adam	0,8209	0,7549	0,066
MLP 2 capas ocultas	activation=relu, hidden_layer_sizes=(100, 50), max_iter=500, solver=adam	0,872	0,7349	0,1371
MLP 3 capas ocultas	activation=relu, hidden_layer_sizes=(100, 50, 25), max_iter=500, solver=adam	0,8843	0,7254	0,1589

La tabla complementa los resultados de prueba con evidencia de validación cruzada. Las mayores brechas se observan en las redes de dos y tres capas y en el árbol de decisión, mientras que SVM mantiene valores de entrenamiento y validación más próximos. Este contraste permite evaluar la selección del modelo no solo por el resultado final en prueba, sino también por su comportamiento durante el proceso de ajuste.

Anexo C. Matrices de confusión y resultados complementarios

Modelo	VN	FP	FN	VP	Accuracy	F1	ROC-AUC
SVM	99	11	20	49	0,8268	0,7597	0,8411
KNN	95	15	18	51	0,8156	0,7556	0,8531
MLP 2 capas ocultas	92	18	19	50	0,7933	0,7299	0,853
MLP 3 capas ocultas	95	15	21	48	0,7989	0,7273	0,858
Naive Bayes	93	17	21	48	0,7877	0,7164	0,8191
Árbol de decisión	99	11	26	43	0,7933	0,6992	0,8073
MLP 1 capa oculta	98	12	26	43	0,7877	0,6935	0,8539

Matrices de confusión - conjunto de prueba

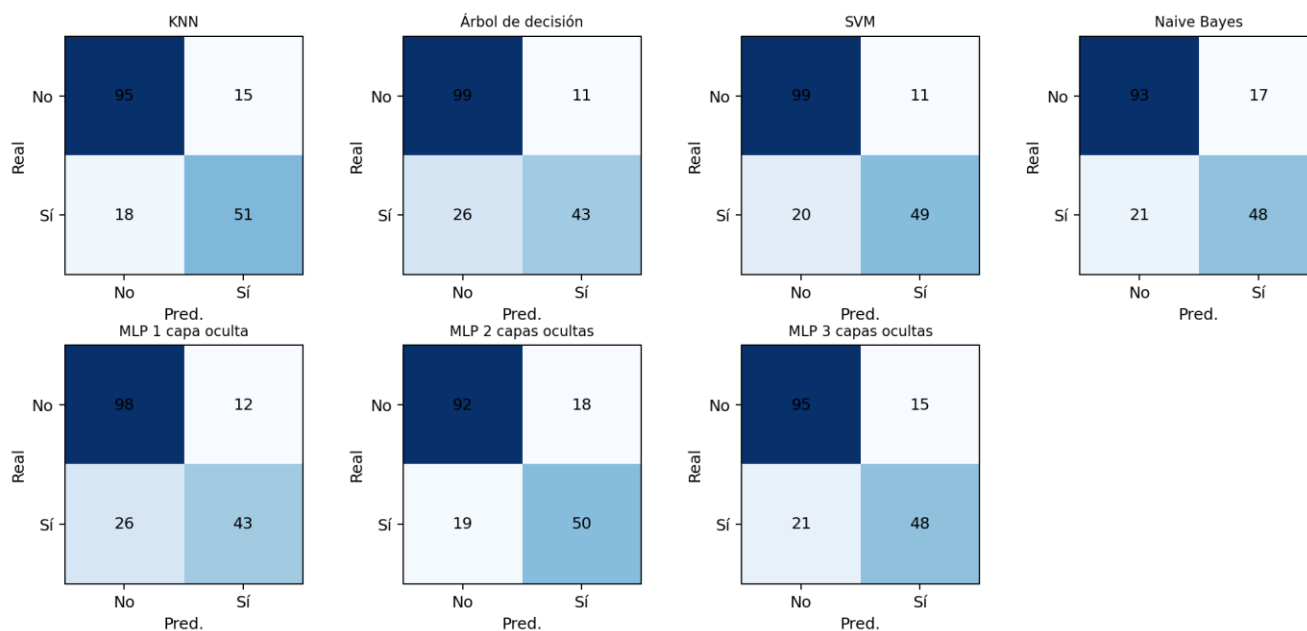


Figura 13. Matrices de confusión consolidadas como evidencia complementaria.

Las matrices de confusión muestran que las diferencias entre modelos no se reducen al número total de aciertos. Por ejemplo, algunos clasificadores disminuyen falsos positivos a costa de aumentar falsos negativos, mientras que otros recuperan más sobrevivientes con una reducción en precisión. Esta evidencia respalda el uso conjunto de precisión, recall y F1 para interpretar el desempeño de la clase positiva.